

My Project

Generated by Doxygen 1.13.2

1 Versijų aprašymai:	1
1.1 v.pradinė:	2
1.1.1 Duomenų nuskaitymas, studentų struktūros sukūrimas, galutinio balo suskaičiavimas	2
1.1.2 su vidurkiu arba mediana. Taip pat ir duomenų išvedimas.	2
1.2 v0.1:	2
1.2.1 Padaryta, kad programa veiktų, nežinant kiek bus namų darbų ar studentų ir realizuota	2
1.2.2 su C masyvu ir vektoriais. Dar padaryta, kad vartotojas galėtų pasirinkti sugeneruoti duomenis.	2
1.3 v0.2:	2
1.3.1 Padaryta, kad duomenis eitų gauti iš tekstinio failo ir kad naudotojas galėtų pasirinkti,	2
1.3.2 pagal ką rūšiuoti. Taip pat reikėjo ištestuoti skirtingo dydžio failus (greitį).	2
1.4 v0.3:	2
1.4.1 Kur tikslinga programoje pradėta naudoti struktūras, daug funkcijų ir duomenų tipų perkelti	2
1.4.2 į skirtingus header ar .cpp files. Pridėtas minimalus išimčių valdymas.	2
1.5 v0.4:	2
1.5.1 Sukurta failų generavimo funkcija, surūšiuoti studentai pagal dvi kategorijas	2
1.5.2 (nuskriaustukai, jei galutinis balas < 5.0 , o kietiakai, jei galutinis balas ≥ 5.0)	2
1.5.3 ir jie išvesti į du skirtingus failus.	2
1.5.4 Atlikta programos veikimo greicio (spartos) analizė.	2
1.6 v1.0:	2
1.6.1 Konteinerių testavimas (sukurtos list ir deque programos ir visų laikai ištestuoti priskaitant vektorius).	2
1.6.2 Optimizuota studentų rūšiavimo (dalijimo) į dvi kategorijas realizacija.	2
1.7 v0.4 Tyrimai	2
1.7.1 1 Tyrimas:	2
1.7.1.1 1) Testuosiu 1 tūkstančio studentų failo sukūrimo ir uždarymo laiką.	3
1.7.1.2 2) Testuosiu 10 tūkstančių studentų failo sukūrimo ir uždarymo laiką.	3
1.7.1.3 3) Testuosiu 100 tūkstančių studentų failo sukūrimo ir uždarymo laiką.	3
1.7.1.4 4) Testuosiu 1 milijono studentų failo sukūrimo ir uždarymo laiką.	3
1.7.1.5 5) Testuosiu 10 milijonų studentų failo sukūrimo ir uždarymo laiką.	4
1.7.2 2 Tyrimas:	4
1.7.2.1 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.	4
1.7.2.2 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.	4
1.7.2.3 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.	4
1.7.2.4 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.	5
1.7.2.5 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.	5
1.8 v1.0 Tyrimai (Konteinerių testavimas)	5
1.8.0.1 Mano nešiojamo kompiuterio (kurį naudoju testavimui) parametrai:	5
1.8.0.2 Testuoju, kai bendrą studentų konteinerį skaidau į du naujus to paties tipo konteinerius: "kietiakai" ir "nuskriaustukai". Techniškai 1 strategija.	6

1.8.0.3 "Rūšiavimas" reiškia studentų rūšiavimą didėjimo tvarką konteineryje su sort. . . .	6
1.8.0.4 "Skirstymas" reiškia studentų skirstymo į dvi grupes/kategorijas (naujų konteinerių su skirtingais studentais kūrimas).	6
1.8.1 1 Tyrimas (Be strategijų)	6
1.8.1.1 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.	6
1.8.1.2 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.	6
1.8.1.3 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.	6
1.8.1.4 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.	7
1.8.1.5 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.	7
1.8.1.6 Taigi, iš konteinerių testavimo matome, kad greičiausiai rūšiuoja (su sort) list konteineris,	7
1.8.1.7 tačiau greičiausiai nuskaitymo ir skirstymo į dvi grupes Vector konteineris.	7
1.9 v1.0 Strategijų tyrimai:	7
1.9.0.1 Kadangi jau padariau pirmą (1) strategiją praeitame testavime, tai dabar testuosiu antrą (2) ir trečią (3) strategiją.	7
1.9.1 2 strategijos tyrimas	7
1.9.1.1 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".	7
1.9.1.2 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".	8
1.9.1.3 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".	8
1.9.1.4 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".	8
1.9.1.5 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".	9
1.9.1.6 Antros strategijos studentų skirstymo į dvi grupes vietoje aš iš tikro panaudojau 3 strategijos std::stable_partition, nes kitaip man neišėjo.	9
1.9.2 3 strategijos tyrimas	9
1.9.2.1 Optimizuojau skirstymą su std::copy_if ir std::partition_copy skaidant bendrą konteinerį į du naujus.	9
1.9.2.2 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).	9
1.9.2.3 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).	10
1.9.2.4 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).	10
1.9.2.5 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).	10
1.9.2.6 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).	10
1.9.2.7 Taigi matome, kad 3 strategija lėtesnė nei 2 ir 1 strategija.	11
1.10 Kaip įdiegti	11

1.11 struct ir class palyginimas, tyrimai:	11
1.11.1 Programos veikimo greičio su -O1 testavimas	11
1.11.2 Programos veikimo greičio su -O2 testavimas	11
1.11.3 Programos veikimo greičio su -O3 testavimas	11
1.11.3.1 Taigi matome, kad struktūros testai buvo greitesni ir -O3 buvo greičiausias beveik visada.	12
1.12 v1.2 Tyrimai (greitis su perdengtais operatoriais ir be)	12
1.12.0.1 Šiuos perdengtus operatorius naudosiu greičio teste:	12
1.12.0.2 Naudosiu pirmą strategiją greičiams matuoti.	12
1.12.0.3 Naudosiu -O3 flag.	12
1.12.0.4 Perdarysiu testus ir be operatorių perdengimo, nes pakeičiau kode bereikalingus <code>std::endl</code> su <code>"\n"</code> .	12
1.12.0.5 Ši kodo vieta leidžia pasirinkti, ar naudoti perdengtus operatorius, ar ne:	12
1.12.1 v1.2 greičio tyrimas su 1 milijonu ir 10 milijonų studentų:	12
1.12.1.1 Paskutinis 1 milijono studentų su operatorių perdengimu testas:	13
1.12.1.2 Paskutinis 10 milijonų studentų su operatorių perdengimu testas:	13
1.12.1.3 Taigi matome, kad testai su operatorių perdengimu buvo vos lėtesni, nei be, bet galima sakyti beveik vienodi.	13
1.13 v1.5 Tyrimai	13
1.13.0.1 Kaip matome, negalime sukurti <code>Zmogus</code> klasės objektų, nes ši klasė yra abstrakti:	13
1.13.0.2 Programa išliko veiksmi naudojant v1.2 logiką.	13
1.13.0.3 Naudosiu pirmą strategiją greičiams matuoti.	13
1.13.0.4 Naudosiu -O3 flag.	13
1.13.0.5 v1.5 greičio tyrimas su 1 milijonu ir 10 milijonų studentų:	13
1.13.0.6 Paskutinis 1 milijono studentų su v1.5 operatorių perdengimu testas:	13
1.13.0.7 Paskutinis 10 milijonų studentų su v1.5 operatorių perdengimu testas:	13
1.13.0.8 Taigi matome, kad v1.5 testai buvo lėtesni nei v1.2 ir be operatorių perdengimo.	13
1.14 v3.0 Vektoriaus kūrimas ir testai	13
1.14.0.1 Reikia kompiliuoti su C++20, nes naudoju <code>construct_at()</code> ir <code>destroy_at()</code> metodus, kad realizuoti vektorių.	13
1.14.1 5 Skirtingų <code>Vector.h</code> funkcijų aprašymas:	13
1.14.1.1 1. <code>void reserve(std::size_t new_cap) :</code>	13
1.14.1.2 2. <code>void resize(std::size_t new_size) :</code>	14
1.14.1.3 3. <code>iterator insert(const_iterator pos, const T& value) :</code>	14
1.14.1.4 4. <code>template <R> void assign_range(R&& rg) :</code>	14
1.14.1.5 5. <code>template <R> void append_range(R&& rg) :</code>	14
1.15 Efektyvumo/spartos analizė - <code>Vector.h</code> VS <code>std::vector</code>	14
1.15.0.1 Užpildysime <code>int</code> elementais su <code>push_back()</code>	14
1.15.1 10 tūkst. elementų:	14
1.15.2 100 tūkst. elementų:	14
1.15.3 1 milijono elementų:	14
1.15.4 10 milijonų elementų:	15
1.15.4.1 100 milijonų elementų	15

1.15.5 Kiek kartų įvyksta konteinerių (Vector ir std::vector) atminties perskirstymai užpildant 100000000 elementų:	15
2 Hierarchical Index	17
2.1 Class Hierarchy	17
3 Class Index	19
3.1 Class List	19
4 File Index	21
4.1 File List	21
5 Class Documentation	23
5.1 Pasirinkimas Class Reference	23
5.2 Studentas Class Reference	23
5.2.1 Member Function Documentation	25
5.2.1.1 Spausdinti()	25
5.3 Timer Class Reference	25
5.4 Vector< T > Class Template Reference	25
5.5 Zmogus Class Reference	27
6 File Documentation	29
6.1 deklaracijos.h	29
6.2 main.h	29
6.3 pasirinkimas.h	30
6.4 studentai.h	30
6.5 timer.h	31
6.6 Vector.h	31
6.7 zmogus.h	37
Index	39

Chapter 1

Versijų aprašymai:

1.1 v.pradinė:

1.1.1 Duomenų nuskaitymas, studentų struktūros sukūrimas, galutinio balo suskaičiavimas

1.1.2 su vidurkiu arba mediana. Taip pat ir duomenų išvedimas.

1.2 v0.1:

1.2.1 Padaryta, kad programa veiktų, nežinant kiek bus namų darbų ar studentų ir realizuota

1.2.2 su C masyvu ir vektoriais. Dar padaryta, kad vartotojas galėtų pasirinkti sugeneruoti duomenis.

1.3 v0.2:

1.3.1 Padaryta, kad duomenis būtų gauti iš tekstinio failo ir kad naudotojas galėtų pasirinkti,

1.3.2 pagal ką rūšiuoti. Taip pat reikėjo ištestuoti skirtingo dydžio failus (greitį).

1.4 v0.3:

1.4.1 Kur tikslinga programoje pradėta naudoti struktūras, daug funkcijų ir duomenų tipų perkelti

1.4.2 į skirtingus header ar .cpp files. Pridėtas minimalus išimčių valdymas.

1.5 v0.4:

1.5.1 Sukurta failų generavimo funkcija, surūšiuoti studentai pagal dvi kategorijas

1.5.2 (nuskriaustukai, jei galutinis balas < 5.0 , o kietiakai, jei galutinis balas ≥ 5.0)

1.5.3 ir jie išvesti į du skirtingus failus.

1.5.4 Atlikta programos veikimo greicio (spartos) analizė.

1.6 v1.0:

1.6.1 Kontaineriu testavimas (sukurtos list ir deque programos ir visu laikai ištestuoti

- Naudoju -O3 vėliavėlę kompiliavimui.

1.7.1.1 1) Testuosiu 1 tūkstančio studentų failo sukūrimo ir uždarymo laiką.

- Kai yra 1 tūkstantis studentų, kiekvienas turi po 10 namų darbų ir vieną egzamino rezultatą.
- Pirmas bandymas:
- 1 tūkst. studentų generavimo laiko testavimų lentelė

1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
0.0213 s	0.0113 s	0.0130 s	0.0109 s	0.0101 s	0.0107 s

1.7.1.2 2) Testuosiu 10 tūkstančių studentų failo sukūrimo ir uždarymo laiką.

- Kai yra 10 tūkstančių studentų, kiekvienas turi po 15 namų darbų ir vieną egzamino rezultatą.
- Pirmas bandymas:
- 10 tūkst. studentų generavimo laiko testavimų lentelė

1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
0.0407 s	0.0408 s	0.0482 s	0.0565 s	0.0385 s	0.0449 s

1.7.1.3 3) Testuosiu 100 tūkstančių studentų failo sukūrimo ir uždarymo laiką.

- Kai yra 100 tūkstančių studentų, kiekvienas turi po 20 namų darbų ir vieną egzamino rezultatą.
- Pirmas bandymas:
- 100 tūkst. studentų generavimo laiko testavimų lentelė

1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
0.2945 s	0.2831 s	0.2908 s	0.2855 s	0.2829 s	0.2873 s

1.7.1.4 4) Testuosiu 1 milijono studentų failo sukūrimo ir uždarymo laiką.

- Kai yra 1 milijonas studentų, kiekvienas turi po 7 namų darbus ir vieną egzamino rezultatą.
- Pirmas bandymas:
- 1 milijono studentų generavimo laiko testavimų lentelė

1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
2.2452 s	2.1820 s	2.1587 s	2.3350 s	2.2060 s	2.2253 s

1.7.1.5 5) Testuosiu 10 milijonų studentų failo sukūrimo ir uždarymo laiką.

- Kai yra 10 milijonų studentų, kiekvienas turi po 5 namų darbus ir vieną egzamino rezultatą.
- Pirmas bandymas:
- 10 milijonų studentų generavimo laiko testavimų lentelė

1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
20.786 s	22.255 s	21.093 s	20.908 s	21.502 s	21.308 s

1.7.2 2 Tyrimas:

1.7.2.1 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.

- Pirmas bandymas:
- 1 tūkst. studentų laiko testavimų lentelė

Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Nuskaitymas	0.0033 s	0.0017 s	0.0029 s	0.0033 s	0.0027 s	0.0027 s
Rūšiavimas	0.0019 s	0.0018 s	0.0020 s	0.0005 s	0.0016 s	0.0015 s
Išvedimas	0.0122 s	0.0058 s	0.0145 s	0.0057 s	0.0137 s	0.0103 s
Vykdydas	0.0335 s	0.0153 s	0.0364 s	0.0183 s	0.0288 s	0.0264 s

1.7.2.2 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.

- Pirmas bandymas:
- 10 tūkst. studentų laiko testavimų lentelė

Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Nuskaitymas	0.0225 s	0.0162 s	0.0147 s	0.0167 s	0.0160 s	0.0172 s
Rūšiavimas	0.0030 s	0.0024 s	0.0026 s	0.0025 s	0.0027 s	0.0026 s
Išvedimas	0.0334 s	0.0350 s	0.0348 s	0.0295 s	0.0434 s	0.0352 s
Vykdydas	0.1045 s	0.1225 s	0.0973 s	0.0902 s	0.1054 s	0.1039 s

1.7.2.3 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.

- Pirmas bandymas:
- 100 tūkst. studentų laiko testavimų lentelė

Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Nuskaitymas	0.1659 s	0.1538 s	0.1474 s	0.1463 s	0.1419 s	0.1510 s
Rūšiavimas	0.0183 s	0.0224 s	0.0144 s	0.0226 s	0.0151 s	0.0185 s
Išvedimas	0.2317 s	0.3066 s	0.2719 s	0.3195 s	0.3220 s	0.2903 s
Vykdymas	0.6698 s	0.8447 s	0.7171 s	0.7885 s	0.8503 s	0.7740 s

1.7.2.4 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.

- Pirmas bandymas:
- 1 milijonų studentų laiko testavimų lentelė

Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Nuskaitymas	0.7563 s	0.8295 s	0.7762 s	0.7925 s	0.7996 s	0.7908 s
Rūšiavimas	0.1044 s	0.1073 s	0.1048 s	0.1444 s	0.1061 s	0.1134 s
Išvedimas	2.4068 s	2.3731 s	2.3550 s	2.2063 s	2.2354 s	2.3153 s
Vykdymas	6.7096 s	5.9877 s	5.9002 s	5.6726 s	5.7194 s	5.9979 s

1.7.2.5 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, išvedimo į du naujus failus ir visos programos veikimo laiką.

- Pirmas bandymas:
- 10 milijonų studentų laiko testavimų lentelė

Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Nuskaitymas	6.9711 s	6.9106 s	6.9154 s	6.8950 s	6.8858 s	6.9155 s
Rūšiavimas	2.0562 s	1.1321 s	1.1247 s	1.3328 s	2.2125 s	1.5716 s
Išvedimas	22.625 s	22.870 s	22.930 s	23.087 s	23.175 s	22.937 s
Vykdymas	57.542 s	58.066 s	59.858 s	59.207 s	57.399 s	58.414 s

1.8 v1.0 Tyrimai (Konteinerių testavimas)

1.8.0.1 Mano nešiojamo kompiuterio (kurį naudoju testavimui) parametrai:

- CPU: 13th Gen Intel(R) Core(TM) i7-13700H 2.40 GHz;
- RAM: 32 GB;
- System type: 64-bit operating system, x64-based processor;
- SSD: 500 GB.

1.8.0.2 Testuoju, kai bendrą studentų konteinerį skaidau į du naujus to paties tipo konteinerius: "kietiakai" ir "nuskriaustukai". Techniškai 1 strategija.

1.8.0.3 "Rūšiavimas" reiškia studentų rūšiavimą didėjimo tvarką konteineryje su sort.

1.8.0.4 "Skirstymas" reiškia studentų skirstymo į dvi grupes/kategorijas (naujų konteinerių su skirtingais studentais kūrimas).

1.8.1 1 Tyrimas (Be strategijų)

1.8.1.1 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.0028 s	0.0053 s	0.0048 s	0.0030 s	0.0056 s	0.0043 s
Vektorius	Rūšiavimas	0.404 ms	0.301 ms	0.231 ms	0.185 ms	0.266 ms	0.277 ms
Vektorius	Skirstymas	0.0072 s	0.0031 s	0.0030 s	0.0022 s	0.0025 s	0.0036 s
List	Nuskaitymas	0.0047 s	0.0064 s	0.0069 s	0.0070 s	0.0048 s	0.0059 s
List	Rūšiavimas	0.247 ms	0.168 ms	0.209 ms	0.157 ms	0.460 ms	0.249 ms
List	Skirstymas	0.0035 s	0.0036 s	0.0035 s	0.0034 s	0.0035 s	0.0034 s
Deque	Nuskaitymas	0.0063 s	0.0059 s	0.0042 s	0.0056 s	0.0031 s	0.0050 s
Deque	Rūšiavimas	0.858 ms	0.839 ms	0.288 ms	0.858 ms	0.464 ms	0.661 ms
Deque	Skirstymas	0.0033 s	0.0033 s	0.0028 s	0.0034 s	0.0034 s	0.0032 s

1.8.1.2 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.0077 s	0.0081 s	0.0072 s	0.0059 s	0.0069 s	0.0071 s
Vektorius	Rūšiavimas	0.0021 s	0.0018 s	0.0018 s	0.0017 s	0.0008 s	0.0016 s
Vektorius	Skirstymas	0.0044 s	0.0045 s	0.0043 s	0.0041 s	0.0041 s	0.0043 s
List	Nuskaitymas	0.0093 s	0.0057 s	0.0092 s	0.0087 s	0.0069 s	0.0080 s
List	Rūšiavimas	0.0011 s	0.0013 s	0.0021 s	0.0012 s	0.0013 s	0.0014 s
List	Skirstymas	0.0031 s	0.0032 s	0.0044 s	0.0039 s	0.0012 s	0.0032 s
Deque	Nuskaitymas	0.0111 s	0.0096 s	0.0112 s	0.0108 s	0.0067 s	0.0099 s
Deque	Rūšiavimas	0.0036 s	0.0038 s	0.0087 s	0.0026 s	0.0028 s	0.0043 s
Deque	Skirstymas	0.0071 s	0.0073 s	0.0052 s	0.0055 s	0.0055 s	0.0061 s

1.8.1.3 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.1626 s	0.1572 s	0.1627 s	0.0181 s	0.1618 s	0.1329 s
Vektorius	Rūšiavimas	0.0210 s	0.0232 s	0.0211 s	0.0233 s	0.0221 s	0.0220 s
Vektorius	Skirstymas	0.0212 s	0.0226 s	0.0181 s	0.0222 s	0.0164 s	0.0206 s
List	Nuskaitymas	0.3337 s	0.3353 s	0.3407 s	0.3309 s	0.2808 s	0.3087 s
List	Rūšiavimas	0.0096 s	0.0098 s	0.0113 s	0.0102 s	0.0105 s	0.0101 s
List	Skirstymas	0.0802 s	0.0875 s	0.0785 s	0.0847 s	0.0875 s	0.0842 s

Deque	Nuskaitymas	0.1742 s	0.1392 s	0.1739 s	0.1724 s	0.1507 s	0.1556 s
Deque	Rūšiavimas	0.0587 s	0.0490 s	0.0589 s	0.0535 s	0.0591 s	0.0572 s
Deque	Skirstymas	0.0343 s	0.0428 s	0.0417 s	0.0441 s	0.0385 s	0.0402 s

1.8.1.4 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.7476 s	0.7022 s	0.6937 s	0.7392 s	0.7067 s	0.7177 s
Vektorius	Rūšiavimas	0.2702 s	0.2695 s	0.2626 s	0.1420 s	0.2713 s	0.2335 s
Vektorius	Skirstymas	0.1249 s	0.1290 s	0.1240 s	0.1626 s	0.1215 s	0.1336 s
List	Nuskaitymas	1.3862 s	1.3127 s	1.3149 s	1.1616 s	1.4077 s	1.3162 s
List	Rūšiavimas	0.1112 s	0.1223 s	0.1149 s	0.1137 s	0.1295 s	0.1214 s
List	Skirstymas	0.2876 s	0.2779 s	0.3099 s	0.2939 s	0.2934 s	0.2936 s
Deque	Nuskaitymas	0.9095 s	0.8445 s	0.8945 s	0.8914 s	0.8429 s	0.8769 s
Deque	Rūšiavimas	0.6868 s	0.6954 s	0.6877 s	0.6779 s	0.6919 s	0.6855 s
Deque	Skirstymas	0.2802 s	0.2794 s	0.2904 s	0.2773 s	0.2757 s	0.2802 s

1.8.1.5 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais.

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	6.5782 s	7.0003 s	7.0122 s	7.2107 s	7.0148 s	6.9005 s
Vektorius	Rūšiavimas	3.4127 s	3.3642 s	3.4868 s	3.7064 s	3.5045 s	3.4744 s
Vektorius	Skirstymas	1.3234 s	1.2963 s	1.3420 s	1.3275 s	1.3065 s	1.3185 s
List	Nuskaitymas	9.2979 s	10.866 s	11.166 s	11.125 s	10.096 s	10.110 s
List	Rūšiavimas	1.4131 s	1.4196 s	1.4264 s	1.4338 s	1.4101 s	1.4260 s
List	Skirstymas	2.4258 s	2.3532 s	2.5715 s	2.4044 s	2.6163 s	2.4660 s
Deque	Nuskaitymas	7.7895 s	8.0954 s	9.2386 s	8.1529 s	8.2182 s	8.3126 s
Deque	Rūšiavimas	8.8716 s	8.3875 s	8.3435 s	8.3998 s	8.4138 s	8.2840 s
Deque	Skirstymas	3.1646 s	3.0068 s	3.0522 s	3.2564 s	3.0442 s	3.1044 s

1.8.1.6 Taigi, iš konteinerių testavimo matome, kad greičiausiai rūšiuoja (su sort) list konteineris,

1.8.1.7 tačiau greičiausiai nuskaitymo ir skirstymo į dvi grupes Vector konteineris.

1.9 v1.0 Strategijų tyrimai:

1.9.0.1 Kadangi jau padariau pirmą (1) strategiją praeitame testavime, tai dabar testuosiu antrą (2) ir trečią (3) strategiją.

1.9.1 2 strategijos tyrimas

- Ši kodo dalis leis sukurti tik "nuskriaustukai" konteinerį:

1.9.1.1 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.0027 s	0.0025 s	0.0014 s	0.0021 s	0.0024 s	0.0022 s
Vektorius	Rūšiavimas	0.285 ms	0.262 ms	0.158 ms	0.190 ms	0.242 ms	0.227 ms
Vektorius	Skirstymas	0.716 ms	0.561 ms	0.397 ms	0.839 ms	0.650 ms	0.632 ms
List	Nuskaitymas	0.0023 s	0.0037 s	0.0022 s	0.0034 s	0.0052 s	0.0034 s
List	Rūšiavimas	0.058 ms	0.079 ms	0.060 ms	0.125 ms	0.162 ms	0.097 ms
List	Skirstymas	0.471 ms	0.456 ms	0.560 ms	0.716 ms	0.869 ms	0.614 ms
Deque	Nuskaitymas	0.0017 s	0.0028 s	0.0024 s	0.0028 s	0.0025 s	0.0024 s
Deque	Rūšiavimas	0.473 ms	0.661 ms	0.352 ms	0.644 ms	0.596 ms	0.545 ms
Deque	Skirstymas	1.120 ms	1.313 ms	0.702 ms	1.232 ms	1.109 ms	1.095 ms

- Paskutinio testo laikai:

1.9.1.2 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.0049 s	0.0036 s	0.0040 s	0.0059 s	0.0055 s	0.0048 s
Vektorius	Rūšiavimas	1.304 ms	0.912 ms	0.789 ms	1.267 ms	0.777 ms	1.009 ms
Vektorius	Skirstymas	0.0012 s	0.0015 s	0.0018 s	0.0031 s	0.0033 s	0.0022 s
List	Nuskaitymas	0.0048 s	0.0045 s	0.0026 s	0.0049 s	0.0052 s	0.0044 s
List	Rūšiavimas	0.538 ms	0.900 ms	1.021 ms	0.563 ms	1.322 ms	0.869 ms
List	Skirstymas	0.0019 s	0.0027 s	0.0028 s	0.0039 s	0.0031 s	0.0029 s
Deque	Nuskaitymas	0.0067 s	0.0090 s	0.0077 s	0.0076 s	0.0080 s	0.0078 s
Deque	Rūšiavimas	1.663 ms	2.440 ms	2.138 ms	2.099 ms	2.840 ms	2.236 ms
Deque	Skirstymas	0.0098 s	0.0160 s	0.0152 s	0.0158 s	0.0159 s	0.0145 s

1.9.1.3 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.1598 s	0.1549 s	0.1551 s	0.1580 s	0.1554 s	0.1566 s
Vektorius	Rūšiavimas	0.0209 s	0.0190 s	0.0187 s	0.0206 s	0.0205 s	0.0199 s
Vektorius	Skirstymas	0.0135 s	0.0111 s	0.0134 s	0.0110 s	0.0134 s	0.0125 s
List	Nuskaitymas	0.3217 s	0.3245 s	0.3367 s	0.3420 s	0.3252 s	0.3300 s
List	Rūšiavimas	0.0066 s	0.0066 s	0.0067 s	0.0077 s	0.0068 s	0.0069 s
List	Skirstymas	0.0634 s	0.0649 s	0.0541 s	0.0636 s	0.0586 s	0.0609 s
Deque	Nuskaitymas	0.1785 s	0.1628 s	0.1727 s	0.1734 s	0.1835 s	0.1742 s
Deque	Rūšiavimas	0.0515 s	0.0534 s	0.0569 s	0.0553 s	0.0508 s	0.0535 s
Deque	Skirstymas	0.0511 s	0.0517 s	0.0514 s	0.0547 s	0.0541 s	0.0526 s

- Paskutinis List testavimas:

1.9.1.4 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	0.7573 s	0.7411 s	0.7576 s	0.7571 s	0.7659 s	0.7558 s
Vektorius	Rūšiavimas	0.2727 s	0.3057 s	0.2724 s	0.2793 s	0.2820 s	0.2824 s
Vektorius	Skirstymas	0.1097 s	0.1190 s	0.1087 s	0.1057 s	0.1105 s	0.1107 s
List	Nuskaitymas	1.3932 s	1.5374 s	1.2863 s	1.2902 s	1.3216 s	1.3657 s
List	Rūšiavimas	0.1271 s	0.1302 s	0.1077 s	0.1184 s	0.1207 s	0.1208 s
List	Skirstymas	0.2959 s	0.3114 s	0.2603 s	0.2622 s	0.2668 s	0.2793 s
Deque	Nuskaitymas	1.0458 s	0.9006 s	0.9008 s	0.9908 s	0.8972 s	0.9470 s
Deque	Rūšiavimas	0.7465 s	0.6755 s	0.6885 s	0.7466 s	0.6666 s	0.7047 s
Deque	Skirstymas	0.6996 s	0.6126 s	0.6361 s	0.7129 s	0.6620 s	0.6646 s

- Paskutinis Deque testavimas:

1.9.1.5 5) Testuosiu 10 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi grupes laiką su skirtingais konteineriais, kuriant tik "nuskriaustukai".

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Nuskaitymas	6.5511 s	6.5167 s	6.5223 s	6.5669 s	6.9504 s	6.6215 s
Vektorius	Rūšiavimas	3.7443 s	3.3765 s	3.3638 s	3.5856 s	3.8797 s	3.5899 s
Vektorius	Skirstymas	1.1952 s	1.1281 s	1.1548 s	1.2840 s	1.2826 s	1.2089 s
List	Nuskaitymas	11.907 s	11.326 s	11.307 s	11.443 s	11.965 s	11.589 s
List	Rūšiavimas	1.5899 s	1.5766 s	1.5652 s	1.5619 s	1.6748 s	1.5937 s
List	Skirstymas	2.7839 s	2.5069 s	2.5780 s	2.6779 s	2.7736 s	2.6641 s
Deque	Nuskaitymas	9.3299 s	8.4383 s	8.3337 s	8.4242 s	8.2841 s	8.5620 s
Deque	Rūšiavimas	9.3203 s	9.3364 s	9.5307 s	9.3106 s	9.2053 s	9.3407 s
Deque	Skirstymas	7.4268 s	7.0624 s	7.5327 s	6.7767 s	6.8542 s	7.1306 s

- Paskutinis Deque testavimas:
- Pastebime, kad rūšiavimas ir skirstymas yra labai neefektyvus procesas su Deque konteineriu.
- Antros strategijos skirstymas dažniausiai yra lėtesnis naudojant List ir ypatingai Deque, bet kartais yra truputi greitesnis su [Vector](#) konteineriu.

1.9.1.6 Antros strategijos studentų skirstymo į dvi grupes vietoje aš iš tikro panaudojau 3 strategijos `std::stable_partition`, nes kitaip man neišėjo.

- Tačiau vis tiek galiu dar labiau pabandyti paoptimizuoti skirstymą su `std::copy_if` ir `std::partition_copy` skaidant bendrą konteinerį į du naujus.

1.9.2 3 strategijos tyrimas

1.9.2.1 Optimizuojau skirstymą su `std::copy_if` ir `std::partition_copy` skaidant bendrą konteinerį į du naujus.

1.9.2.2 1) Testuosiu 1 tūkstančio studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Skirstymas	0.0004 s	0.0005 s	0.0019 s	0.0013 s	0.0035 s	0.0015 s
List	Skirstymas	0.0018 s	0.0027 s	0.0022 s	0.0023 s	0.0025 s	0.0023 s
Deque	Skirstymas	0.0007 s	0.0019 s	0.0016 s	0.0026 s	0.0030 s	0.0020 s

1.9.2.3 2) Testuosiu 10 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Skirstymas	0.0014 s	0.0049 s	0.0034 s	0.0025 s	0.0032 s	0.0031 s
List	Skirstymas	0.0023 s	0.0034 s	0.0035 s	0.0032 s	0.0026 s	0.0030 s
Deque	Skirstymas	0.0045 s	0.0115 s	0.0065 s	0.0075 s	0.0101 s	0.0080 s

- Paskutinis Deque testavimas:

1.9.2.4 3) Testuosiu 100 tūkstančių studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Skirstymas	0.0259 s	0.0262 s	0.0247 s	0.0249 s	0.0245 s	0.0252 s
List	Skirstymas	0.1446 s	0.1421 s	0.1441 s	0.1394 s	0.1492 s	0.1439 s
Deque	Skirstymas	0.0660 s	0.0740 s	0.0656 s	0.0637 s	0.0650 s	0.0669 s

- Paskutinis Deque testavimas:

1.9.2.5 4) Testuosiu 1 milijono studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Skirstymas	0.1916 s	0.1916 s	0.1881 s	0.2025 s	0.1962 s	0.1940 s
List	Skirstymas	0.6241 s	0.6367 s	0.6788 s	0.5664 s	0.6010 s	0.6214 s
Deque	Skirstymas	0.5776 s	0.6110 s	0.6007 s	0.6209 s	0.6194 s	0.6059 s

- Paskutinis Deque testavimas:

1.9.2.6 5) Testuosiu 10 milijonų studentų failo nuskaitymo, rūšiavimo, skirstymo į dvi naujas grupes laiką su skirtingais konteineriais (3 strategija).

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
Vektorius	Skirstymas	2.2186 s	2.2722 s	2.3623 s	2.4343 s	2.4951 s	2.3565 s
List	Skirstymas	5.9380 s	5.1058 s	5.4631 s	5.3478 s	5.2896 s	5.4289 s
Deque	Skirstymas	7.6150 s	8.5334 s	6.7577 s	6.7577 s	6.4516 s	7.2231 s

- Paskutinis Deque testavimas:

1.9.2.7 Taigi matome, kad 3 strategija lėtesnė nei 2 ir 1 strategija.

1.10 Kaip įdiegti

1) Pasirinkite [Vector](#) ar List ar Deque programos aplanką, naudojant šias komandas terminale:

- cd [Vector](#) Arba cd List Arba cd Deque 2) Sukurkite build aplanką ir įeikite į jį, naudojant šias komandas terminale:
- mkdir build
- cd build 3) Paleiskite cmake, naudojant šias komandas terminale:
- cmake .. 4) Išeikite iš aplankalo ir sukompiliuokite programą, naudojant šias komandas terminale:
- cd ..
- cmake --build build 5) Programa bus build/debug aplanke su pavadinimu "v1.0_uzduotis.exe". 6) Tada galėsite paleisti executable.

1.11 struct ir class palyginimas, tyrimai:

* Palyginsime greičius su -O1, -O2 ir -O3 vėliavėlėmis.
 * Testavimui naudosime tik 1 ir 10 milijonų studentų failus.
 * Matuosime visos programos veikimo laiką, neįskaitant vartotojo duomenų įvesties laiko.
 * Naudosiu pirmą (1) strategiją, kad testuoti greičius.

1.11.1 Programos veikimo greičio su -O1 testavimas

Stud.sk/Duomenų tipas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
1 milijonas / class	6.8223 s	6.9257 s	6.8526 s	6.5403 s	6.6768 s	6.7635 s
1 milijonas / struct	6.1602 s	6.1610 s	6.0750 s	6.0470 s	6.0996 s	6.1086 s
10 milijonų / class	74.869 s	77.282 s	76.083 s	76.903 s	75.671 s	76.162 s
10 milijonų / struct	62.441 s	57.489 s	62.013 s	61.107 s	60.523 s	60.715 s

Klasės .exe failo dydis: 3.162 KB. Struktūros .exe failo dydis: 3.163 KB.

1.11.2 Programos veikimo greičio su -O2 testavimas

Stud.sk/Duomenų tipas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
1 milijonas / class	6.5051 s	6.3632 s	6.4580 s	6.5346 s	6.5438 s	6.4809 s
1 milijonas / struct	5.9997 s	5.9901 s	5.7390 s	6.0790 s	5.8582 s	5.9332 s
10 milijonų / class	73.144 s	72.125 s	71.557 s	73.392 s	73.284 s	72.700 s
10 milijonų / struct	59.897 s	61.172 s	60.671 s	61.792 s	60.832 s	60.873 s

Klasės .exe failo dydis: 3.164 KB. Struktūros .exe failo dydis: 3.168 KB.

1.11.3 Programos veikimo greičio su -O3 testavimas

Stud.sk/Duomenų tipas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
1 milijonas / class	6.3353 s	6.6236 s	6.3143 s	6.6149 s	6.6479 s	6.5072 s
1 milijonas / struct	6.0467 s	6.1405 s	5.5989 s	5.9337 s	5.7202 s	5.8880 s
10 milijonų / class	72.901 s	72.158 s	73.451 s	74.351 s	74.699 s	73.512 s
10 milijonų / struct	59.576 s	57.394 s	58.490 s	57.158 s	57.183 s	57.960 s

Klasės .exe failo dydis: 3.196 KB. Struktūros .exe failo dydis: 3.197 KB.

1.11.3.1 Taigi matome, kad struktūros testai buvo greitesni ir -O3 buvo greičiausias beveik visada.

1.12 v1.2 Tyrimai (greitis su perdengtais operatoriais ir be)

1.12.0.1 Šiuos perdengtus operatorius naudosiu greičio teste:

- įvesties;
- failo nuskaitymo;
- išvesties;
- failo išvesties.

1.12.0.2 Naudosiu pirmą strategiją greičiams matuoti.

1.12.0.3 Naudosiu -O3 flag.

1.12.0.4 Perdarysiu testus ir be operatorių perdengimo, nes pakeičiau kode bereikalingus std::endl su "\n".

1.12.0.5 Ši kodo vieta leidžia pasirinkti, ar naudoti perdengtus operatorius, ar ne:

1.12.1 v1.2 greičio tyrimas su 1 milijonu ir 10 milijonų studentų:

Stud.sk/Operatorius	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
1 milijonas / be oper.	2.1869 s	2.2442 s	2.3407 s	2.2601 s	2.5255 s	2,3115 s
1 milijonas / su oper.	2.2164 s	2.2581 s	2.2541 s	2.2703 s	2.1906 s	2.3244 s
10 milijonų / be oper.	23.347 s	23.189 s	23.001 s	23.798 s	23.887 s	23.444 s
10 milijonų / su oper.	23.652 s	24.038 s	24.068 s	23.806 s	23.821 s	23.877 s

1.12.1.1 Paskutinis 1 milijono studentų su operatorių perdengimu testas:

1.12.1.2 Paskutinis 10 milijonų studentų su operatorių perdengimu testas:

1.12.1.3 Taigi matome, kad testai su operatorių perdengimu buvo vos lėtesni, nei be, bet galima sakyti beveik vienodi.

1.13 v1.5 Tyrimai

1.13.0.1 Kaip matome, negalime sukurti Zmogus klasės objektų, nes ši klasė yra abstrakti:

1.13.0.2 Programa išliko veiksmi naudojant v1.2 logiką.

1.13.0.3 Naudosiu pirmą strategiją greičiams matuoti.

1.13.0.4 Naudosiu -O3 flag.

1.13.0.5 v1.5 greičio tyrimas su 1 milijonu ir 10 milijonų studentų:

Stud.sk/Operatorius	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
1 milijonas / be oper.	2.1869 s	2.2442 s	2.3407 s	2.2601 s	2.5255 s	2,3115 s
1 milijonas / su oper. v1.2	2.2164 s	2.2581 s	2.2541 s	2.2703 s	2.1906 s	2.3244 s
1 milijonas / su oper. v1.5	3.0894 s	2.9806 s	3.0065 s	3.2908 s	2.4853 s	2.9705 s
10 milijonų / be oper.	23.347 s	23.189 s	23.001 s	23.798 s	23.887 s	23.444 s
10 milijonų / su oper. v1.2	23.652 s	24.038 s	24.068 s	23.806 s	23.821 s	23.877 s
10 milijonų / su oper. v1.5	27.407 s	25.158 s	25.121 s	24.679 s	24.695 s	25.412 s

1.13.0.6 Paskutinis 1 milijono studentų su v1.5 operatorių perdengimu testas:

1.13.0.7 Paskutinis 10 milijonų studentų su v1.5 operatorių perdengimu testas:

1.13.0.8 Taigi matome, kad v1.5 testai buvo lėtesni nei v1.2 ir be operatorių perdengimo.

1.14 v3.0 Vektoriaus kūrimas ir testai

1.14.0.1 Reikia kompiliuoti su C++20, nes naudoju `construct_at()` ir `destroy_at()` metodus, kad realizuoti vektorių.

1.14.1 5 Skirtingų `Vector.h` funkcijų aprašymas:

1.14.1.1 1. `void reserve(std::size_t new_cap) :`

- Užtikrina, kad vidinis masyvas turėtų bent `new_cap` vietų.
- Jeigu reikia, alokuoja naują bloką, perkelia (move) egzistuojančius elementus ir atnaujiną `_capacity`, bet nedidina `_size`.

1.14.1.2 2. void resize(std::size_t new_size) :

- Pakeičia logišką vektoriaus ilgį `_size`. Mažinant – sunaikina (`std::destroy_at`) perteklinius objektus;
- didinant – sukuria naujų (default-konstruoja) ir prireikus išskviečia `reserve`, kad atmintis tilptų.

1.14.1.3 3. iterator insert(const_iterator pos, const T& value) :

- Įterpia vieną elementą nurodytoje pozicijoje.
- Prireikus plečia talpą, tada perkelia visus elementus į dešinę ir galiausiai konstruoja naują kopiją `value`; grąžina iteratorių į įterptą elementą.

1.14.1.4 4. template <R> void assign_range(R&& rg) :

- C++20 diapazonų (Ranges) pagrindu pakeičia visą vektoriaus turinį nauju intervalu.
- Naudoja `std::ranges::distance`, `begin` ir `static_assert`, kad kompiliavimo metu tikrintų, ar `R` yra bent jau input-range ir kad iš jo elementų galima konstruoti `T`.

1.14.1.5 5. template <R> void append_range(R&& rg) :

- Prideda kiekvieną elemento nuorodą iš Ranges intervalo į vektoriaus galą.
- Dinamiškai perskaičiuoja vietos poreikį, prireikus išskviečia `reserve`, o kiekvieną naują objektą sukonstruoja vietoje (`std::construct_at`) su `std::forward`.

1.15 Efektyvumo/spartos analizė - Vector.h VS std::vector**1.15.0.1 Užpildysime int elementais su push_back()****1.15.1 10 tūkst. elementų:**

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
std::vector	Užpildymas	6.1e-05 s	9.1e-05 s	5.4e-05 s	4.9e-05 s	7.1e-05 s	6.3e-05 s
Vector.h	Užpildymas	4.4e-05 s	7e-05 s	4.6e-05 s	6.9e-05 s	4.5e-05 s	5.4e-05 s

1.15.2 100 tūkst. elementų:

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
std::vector	Užpildymas	0.0004 s	0.0005 s	0.0005 s	0.0003 s	0.0005 s	0.0004 s
Vector.h	Užpildymas	0.0002 s	0.0003 s	0.0001 s	0.0001 s	0.0002 s	0.0002 s

1.15.3 1 milijono elementų:

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
std::vector	Užpildymas	0.0030 s	0.0027 s	0.0021 s	0.0026 s	0.0030 s	0.0027 s
Vector.h	Užpildymas	0.0023 s	0.0024 s	0.0027 s	0.0022 s	0.0024 s	0.0024 s

1.15.4 10 milijonų elementų:

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
std::vector	Užpildymas	0.0277 s	0.0207 s	0.0207 s	0.0224 s	0.0235 s	0.0230 s
Vector.h	Užpildymas	0.0270 s	0.0319 s	0.0318 s	0.0321 s	0.0344 s	0.0314 s

1.15.4.1 100 milijonų elementų

Konteineris	Matavimas	1 testas	2 testas	3 testas	4 testas	5 testas	Vidurkis
std::vector	Užpildymas	0.1976 s	0.1872 s	0.1980 s	0.1894 s	0.2143 s	0.1639 s
Vector.h	Užpildymas	0.2225 s	0.1933 s	0.2078 s	0.1973 s	0.2032 s	0.1718 s

1.15.5 Kiek kartų įvyksta konteinerių (Vector ir std::vector) atminties perskirstymai užpildant 100000000 elementų:

- Atsakymas - 28

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Pasirinkimas	23
Timer	25
Vector< T >	25
Zmogus	27
Studentas	23

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Pasirinkimas	23
Studentas	23
Timer	25
Vector< T >	25
Zmogus	27

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

deklaracijos.h	29
main.h	29
pasirinkimas.h	30
studentai.h	30
timer.h	31
Vector.h	31
zmogus.h	37

Chapter 5

Class Documentation

5.1 Pasirinkimas Class Reference

Public Attributes

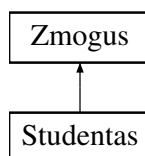
- int **m** =0
- int **n** =0
- int **vid_ar_med** =0
- int **irasyti_dar_studentu** =3
- int **rasyti_daugiau** =5
- int **kaip_gauti_duomenis** =0
- int **pradeti_baigti** =0
- int **ar_generuoti_failus** =0
- int **stud_kiekis** =0
- int **kur_isvesti** =0
- int **koks_konteineris** =0
- int **koks_file_pavadinimas** =0
- int **testai** =0
- int **ar naudoti_klases_operatorius** =0

The documentation for this class was generated from the following file:

- pasirinkimas.h

5.2 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (const std::string &v, const std::string &p, int egz, const [Vector](#)< int > &nd_vec)
- **Studentas** (const [Studentas](#) &naujas)
- **Studentas** & **operator=** (const [Studentas](#) &naujas)
- **Studentas** ([Studentas](#) &&naujas) noexcept
- **Studentas** & **operator=** ([Studentas](#) &&naujas) noexcept
- int **GetEgz** () const
- [Vector](#)< int > **GetNd** () const
- double **GetGalutinis** () const
- void **SetEgz** (int egz)
- void **SetNd** (int &nd)
- void **SetNd** (const [Vector](#)< int > &nd)
- void **SetGalutinis** (double galutinis)
- void **NdClear** ()
- void **SetAllNd** ([Vector](#)< int > &nd)
- void **CalcVid** ()
- void **CalcMed** ()
- void **Spausdinti** (std::ostream &) const override

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (string vardas, string pavarde)
- string **GetVardas** () const
- string **GetPavarde** () const
- void **SetVardas** (string vardas)
- void **SetPavarde** (string pavarde)
- int **GetVardasSize** () const
- int **GetPavardeSize** () const

Static Public Member Functions

- static void **Rikiavimas** ([Vector](#)< [Studentas](#) > &studentai, int rikiavimas)

Friends

- istream & **operator>>** (istream &is, [Studentas](#) &studentas)
- ifstream & **operator>>** (ifstream &is, [Studentas](#) &studentas)
- ostream & **operator<<** (ostream &os, const [Studentas](#) &studentas)
- ofstream & **operator<<** (ofstream &os, const [Vector](#)< [Studentas](#) > &studentai)
- ostream & **operator<<** (ostream &os, const [Vector](#)< [Studentas](#) > &studentai)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string **vardas**
- std::string **pavarde**

5.2.1 Member Function Documentation

5.2.1.1 Spausdinti()

```
void Studentas::Spausdinti (
    std::ostream & os) const [override], [virtual]
```

Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- studentai.h
- studentai.cpp

5.3 Timer Class Reference

Public Member Functions

- void **reset** ()
- double **elapsed** () const

The documentation for this class was generated from the following files:

- timer.h
- timer.cpp

5.4 Vector< T > Class Template Reference

Public Member Functions

- **Vector** (size_t dydis)
- **Vector** (std::initializer_list< T > il)
- **Vector** (const [Vector](#)< T > &naujas)
- **Vector** ([Vector](#)< T > &&naujas) noexcept
- [Vector](#)< T > & **operator=** (const [Vector](#)< T > &naujas)
- [Vector](#)< T > & **operator=** ([Vector](#)< T > &&naujas) noexcept
- T & **operator[]** (size_t index)
- const T & **operator[]** (size_t index) const
- T & **at** (size_t indeksas)
- const T & **at** (size_t indeksas) const
- T & **front** ()
- const T & **front** () const
- T & **back** ()
- const T & **back** () const
- size_t **max_size** () const
- size_t **size** () const noexcept
- size_t **capacity** () const noexcept
- bool **empty** () const noexcept
- void **reserve** (size_t new_capacity)

- void **shrink_to_fit** ()
- void **resize** (size_t new_size)
- void **push_back** (const T &new_value)
- void **push_back** (T &&new_value)
- void **pop_back** ()
- void **clear** ()
- void **swap** (Vector< T > &naujas)
- void **sort** ()
- void **assign** (size_t count, const T &value)
- iterator **end** () noexcept
- const_iterator **end** () const
- const_iterator **cend** () const noexcept
- iterator **begin** () noexcept
- const_iterator **begin** () const noexcept
- const_iterator **cbegin** () const noexcept
- reverse_iterator **rbegin** () noexcept
- const_reverse_iterator **rbegin** () const noexcept
- reverse_iterator **rend** () noexcept
- const_reverse_iterator **rend** () const noexcept
- const_reverse_iterator **crbegin** () const noexcept
- const_reverse_iterator **crend** () const noexcept
- iterator **insert** (const_iterator pos, const T &value)
- iterator **insert** (const_iterator pos, T &&value)
- T * **data** () noexcept
- const T * **data** () const
- template<typename... Args>
iterator **emplace** (const_iterator pos, Args &&... args)
- template<typename... Args>
void **emplace_back** (Args &&... args)
- template<typename InputIt>
iterator **insert_range** (const_iterator pos, InputIt first, InputIt last)
- template<typename R>
constexpr void **assign_range** (R &&rg)
- allocator_type **get_allocator** () const
- template<typename R>
constexpr void **append_range** (R &&rg)
- iterator **erase** (const_iterator pos)
- iterator **erase** (const_iterator first, const_iterator last)

Friends

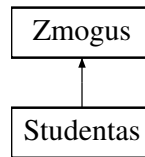
- bool **operator==** (const Vector< T > &a, const Vector< T > &b)
- bool **operator!=** (const Vector< T > &a, const Vector< T > &b)
- template<class U>
std::ostream & **operator<<** (std::ostream &os, const Vector< U > &vec)

The documentation for this class was generated from the following file:

- Vector.h

5.5 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (string vardas, string pavarde)
- string **GetVardas** () const
- string **GetPavarde** () const
- void **SetVardas** (string vardas)
- void **SetPavarde** (string pavarde)
- int **GetVardasSize** () const
- int **GetPavardeSize** () const
- virtual void **Spausdinti** (std::ostream &os) const =0

Protected Attributes

- std::string **vardas**
- std::string **pavarde**

The documentation for this class was generated from the following files:

- zmogus.h
- zmogus.cpp

Chapter 6

File Documentation

6.1 deklaracijos.h

```
00001 #ifndef DEKLARACIJOS_H
00002 #define DEKLARACIJOS_H
00003
00004 #include "studentai.h"
00005 #include "timer.h"
00006 #include "pasirinkimas.h"
00007
00008 extern Vector<Studentas> studentai, kietiakai, nuskriaustukai;
00009 extern Vector<string> vardai, pavardes;
00010 extern Studentas st;
00011 extern Pasirinkimas p;
00012 extern int vard_size, pav_size;
00013 extern string randomVyrVardai[20];
00014 extern string randomMotVardai[20];
00015 extern string randomVyrPavarde[20];
00016 extern string randomMotPavarde[20];
00017 void ivedimas1(int& pasirinkimas, string a, string b, int decider, int o1, int o2);
00018 void file_generavimas(int &n);
00019 void ivedimas2(int& pasirinkimas);
00020 void ivedimas3(int& pasirinkimas);
00021 void data_input(int &vard_size, int &pav_size, Pasirinkimas p, Studentas st, Vector<Studentas>&
    studentai, Vector<string> vardai, Vector<string> pavardes, string randomVyrVardai[], string
    randomMotVardai[], string randomVyrPavarde[], string randomMotPavarde[],
    time_point<high_resolution_clock> &start);
00022 void isvedimasFun(const Pasirinkimas &p, const Vector<Studentas>& studentai, string failo_pav, int
    vard_size, int pav_size);
00023 void file_skaitymas(int &vard_size, int &pav_size, Pasirinkimas p, Studentas st, Vector<Studentas>&
    studentai, Vector<string> vardai, Vector<string> pavardes, time_point<high_resolution_clock> &start,
    int *max_vardas_size, int *max_pavarde_size);
00024 void RunTests();
00025 void RunVectorTests();
00026 void spartos_test();
00027 #endif
```

6.2 main.h

```
00001 #ifndef MAIN_H
00002 #define MAIN_H
00003
00004 //Su vektoriais:
00005 #include <iostream>
00006 #include <string>
00007 #include <iomanip>
00008 #include <algorithm>
00009 #include <limits>
00010 #include <ctime>
00011 #include <fstream>
00012 #include <cstdlib>
00013 #include <sstream>
00014 #include <chrono>
00015 #include <cstdint>
00016 #include "Vector.h"
00017
```

```

00018 using std::cout; using std::cin; using std::endl; using std::string;
00019 using std::fixed; using std::setprecision; using std::setw; using std::left;
00020 using std::sort; using std::numeric_limits; using std::streamsize;
00021 using std::ifstream; using std::ofstream;
00022 using std::stringstream; using std::getline; using std::cerr; using std::runtime_error; using
std::chrono::high_resolution_clock;
00023 using std::chrono::time_point; using std::chrono::duration; using std::chrono::duration_cast; using
std::exit; using std::rand;
00024 using std::to_string; using std::ostream; using std::move; using std::istream;
00025
00026 #endif

```

6.3 pasirinkimas.h

```

00001 #ifndef PASIRINKIMAS_H
00002 #define PASIRINKIMAS_H
00003
00004 class Pasirinkimas{
00005     public:
00006         int m=0, n=0, vid_ar_med=0, irasyti_dar_studentu=3, rasyti_daugiau=5, kaip_gauti_duomenis=0,
pradeti_baigti=0,
00007         ar_generuoti_failus=0, stud_kiekis=0, kur_istesti=0, koks_konteineris=0, koks_file_pavadinimas=0,
testai=0,
00008         ar_naudoti_klases_operatorius=0;
00009 };
00010 #endif

```

6.4 studentai.h

```

00001 #ifndef STUDENTAI_H
00002 #define STUDENTAI_H
00003
00004 #include "zmogus.h"
00005 #include <stdexcept>
00006
00007
00008 class Studentas : public Zmogus{
00009
00010     private:
00011         int egz;
00012         Vector<int> nd;
00013         double galutinis;
00014
00015     public:
00016         Studentas() : Zmogus(), egz(0), nd {}, galutinis(0){} // Default konstruktorius
00017         Studentas(const std::string& v, const std::string& p, int egz, const Vector<int>& nd_vec); //
Konstruktorius
00018         ~Studentas(); // Destruktorius
00019
00020         Studentas(const Studentas& naujas); //Copy konstruktorius
00021         Studentas& operator=(const Studentas& naujas); //Copy priskyrimo operatorius
00022         Studentas(Studentas&& naujas) noexcept; //Move konstruktorius
00023         Studentas& operator=(Studentas&& naujas) noexcept; //Move priskyrimo operatorius
00024         friend istream& operator<>(istream& is, Studentas& studentas); // Ivedimo operatorius
00025         friend ifstream& operator<>(ifstream& is, Studentas& studentas); // Ivedimo operatorius is failo
00026         friend ostream& operator<>(ostream& os, const Studentas& studentas); // Isvedimo operatorius
00027         friend ofstream& operator<>(ofstream& os, const Vector<Studentas>& studentai); // Isvedimo
operatorius i faila
00028         friend ostream& operator<>(ostream& os, const Vector<Studentas>& studentai); // Isvedimo
operatorius i ekrana
00029
00030         int GetEgz() const; // Grazina egzamino bala
00031         Vector<int> GetNd() const; // Grazina namu darbu balus
00032         double GetGalutinis() const; // Grazina galutini bala
00033         void SetEgz(int egz); // Nustato egzamino bala
00034         void SetNd(int &nd); // Nustato namu darbu balus
00035         void SetNd(const Vector<int>& nd); // Nustato namu darbu balus
00036         void SetGalutinis(double galutinis); // Nustato galutini bala
00037         void NdClear(); // Istrina namu darbu rezultatus
00038         void SetAllNd(Vector<int>& nd); // Prideda visus namu darbu balus i vektoriui
00039         void CalcVid(); // Apskaiciuoja galutini bala pagal vidurki
00040         void CalcMed(); // Apskaiciuoja galutini bala pagal mediana
00041         static void Rikiavimas(Vector<Studentas>& studentai, int rikiavimas); // Rikiuoja studentus pagal
varda, pavarde arba galutini bala
00042         void Spausdinti(std::ostream&) const override; // Isvedimo funkcija
00043 };
00044
00045 #endif

```

6.5 timer.h

```

00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include "main.h"
00005
00006 class Timer {
00007     private:
00008         using hrClock = high_resolution_clock;
00009         using durationDouble = duration<double>;
00010         std::chrono::time_point<hrClock> start;
00011     public:
00012         Timer();
00013         void reset();
00014         double elapsed() const;
00015 };
00016 #endif

```

6.6 Vector.h

```

00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <ranges>
00005 #include <iterator>
00006 #include <utility> // for std::move
00007 #include <type_traits>
00008 #include <concepts>
00009 #include <memory>
00010 #include <stdexcept>
00011 #include <algorithm>
00012 #include <limits>
00013
00014 template <typename T>
00015 class Vector{
00016     size_t _size;
00017     size_t _capacity;
00018     T* _data;
00019
00020     static T* allocate(std::size_t n) {
00021         return n ? new T[n] : nullptr;
00022     }
00023     static void deallocate(T* p) {
00024         delete [] p;
00025     }
00026
00027     using iterator = T*;
00028     using const_iterator = const T*;
00029     using reverse_iterator = std::reverse_iterator<iterator>;
00030     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00031     using allocator_type = std::allocator<T>;
00032     std::allocator<T> alloc;
00033
00034     public:
00035
00036     Vector(): _size(0), _capacity(0), _data(nullptr) {}; // Konstruktorius
00037
00038     Vector(size_t dydis) : _size(dydis), _capacity(dydis), _data(dydis ? new T[dydis] : nullptr) { //
00039         Konstruktorius su dydžiu
00040         for (size_t i = 0; i < _size; ++i) {
00041             _data[i] = T{}; // Inicializuojame elementus
00042         }
00043     }
00044
00045     Vector(std::initializer_list<T> il)
00046         : _size(il.size()),
00047         _capacity(il.size()),
00048         _data(il.size() ? new T[il.size()] : nullptr)
00049     {
00050         std::uninitialized_copy(il.begin(), il.end(), _data);
00051     }
00052
00053     Vector(const Vector<T>& naujas) : _data(naujas._capacity ? new T[naujas._capacity] : nullptr),
00054         _size(naujas._size), _capacity(naujas._capacity) // Copy constructor
00055     {
00056         for (size_t i = 0; i < _size; ++i) {
00057             std::construct_at(_data+i, naujas._data[i]);
00058         }
00059     }

```

```

00060 Vector(Vector<T>&& naujas) noexcept: _data(naujas._data), _size(naujas._size),
    _capacity(naujas._capacity) // Move constructor
00061 {
00062     naujas._data = nullptr;
00063     naujas._size = 0;
00064     naujas._capacity = 0;
00065 }
00066
00067 ~Vector() { // Destruktorius
00068     for (size_t i = 0; i < _size; ++i)
00069         std::destroy_at(_data + i); // sunaikina visus T objektus
00070
00071     delete[] _data;
00072 }
00073
00074 Vector<T>& operator=(const Vector<T>& naujas) // Copy priskyrimo operatorius
00075 {
00076     if (this == &naujas)
00077         return *this;
00078
00079     T* new_data = new T[naujas._capacity];
00080     try {
00081         for (size_t i = 0; i < naujas._size; ++i)
00082             new_data[i] = naujas._data[i];
00083     } catch (...) {
00084         delete[] new_data;
00085         throw;
00086     }
00087
00088     delete[] _data; // Istriname sena atminties bloka
00089     _data = new_data; // Sukuriame nauja atminties bloka
00090     _size = naujas._size;
00091     _capacity = naujas._capacity;
00092     return *this;
00093 }
00094
00095
00096 Vector<T>& operator=(Vector<T>&& naujas) noexcept
00097 {
00098     if (this == &naujas)
00099         return *this;
00100
00101     //Jeigu this != &naujas
00102     clear();
00103     delete[] _data;
00104
00105     _data = naujas._data;
00106     _size = naujas._size;
00107     _capacity = naujas._capacity;
00108
00109     naujas._data = nullptr;
00110     naujas._size = 0;
00111     naujas._capacity = 0;
00112
00113     return *this;
00114 }
00115
00116 T& operator[](size_t index)
00117 {
00118     return _data[index];
00119 }
00120
00121 const T& operator[](size_t index) const
00122 {
00123     return _data[index];
00124 }
00125
00126 T& at(size_t indeksas)
00127 {
00128     if (indeksas < 0 || indeksas >= _size)
00129         throw std::out_of_range("Vector::at(): indeksas uz ribu");
00130     else
00131         return _data[indeksas];
00132 }
00133
00134 const T& at(size_t indeksas) const
00135 {
00136     if (indeksas < 0 || indeksas >= _size)
00137         throw std::out_of_range("Vector::at(): indeksas uz ribu");
00138     else
00139         return _data[indeksas];
00140 }
00141
00142 T& front()
00143 {
00144     return _data[0];
00145 }

```

```

00146
00147 const T& front() const
00148 {
00149     return _data[0];
00150 }
00151
00152 T& back()
00153 {
00154     return _data[_size-1];
00155 }
00156
00157 const T& back() const
00158 {
00159     return _data[_size-1];
00160 }
00161
00162 size_t max_size() const
00163 {
00164     return std::numeric_limits<size_t>::max() / sizeof(T); // dalinti iš sizeof(T), nes tiek elementų
    tilptų į maksimalų baitų kiekį.
00165 }
00166
00167 size_t size() const noexcept
00168 {
00169     return _size;
00170 }
00171
00172 size_t capacity() const noexcept
00173 {
00174     return _capacity;
00175 }
00176
00177 bool empty() const noexcept
00178 {
00179     if(_size == 0)
00180         return true;
00181     else
00182         return false;
00183 }
00184
00185 void reserve(size_t new_capacity)
00186 {
00187     if(new_capacity > max_size())
00188         throw std::length_error("Vector reserve() per didelį");
00189     else if(new_capacity > _capacity)
00190     {
00191         T* new_data = new T[new_capacity];
00192         for(size_t i=0; i<_size; i++)
00193         {
00194             new_data[i] = std::move(_data[i]);
00195         }
00196         delete[] _data;
00197         _data = new_data;
00198         _capacity = new_capacity;
00199     }
00200 }
00201
00202 void shrink_to_fit()
00203 {
00204     if (_size == 0)
00205     {
00206         delete[] _data; _data=nullptr; _capacity=0; return;
00207     }
00208
00209     if(_capacity == _size)
00210         return;
00211
00212     T* new_data = new T[_size];
00213     for(size_t i = 0; i < _size; i++)
00214     {
00215         new_data[i] = std::move(_data[i]);
00216     }
00217     delete[] _data;
00218     _data = new_data;
00219     _capacity = _size;
00220 }
00221
00222 void resize(size_t new_size)
00223 {
00224     if (new_size == _size) return;
00225
00226     if (new_size < _size)
00227     {
00228         for (size_t i = new_size; i < _size; ++i)
00229             std::destroy_at(_data + i);
00230
00231         _size = new_size;

```

```

00232         return;
00233     }
00234
00235     if (new_size <= _capacity)
00236     {
00237         for (size_t i = _size; i < new_size; ++i)
00238             std::construct_at(_data + i);
00239
00240         _size = new_size;
00241         return;
00242     }
00243
00244     reserve(new_size);          // reserve() pasirūpins _capacity & kopijavimu
00245
00246     for (size_t i = _size; i < new_size; ++i)
00247         std::construct_at(_data + i);
00248
00249     _size = new_size;
00250 }
00251
00252 void push_back(const T& new_value)
00253 {
00254     if(_size >= _capacity)
00255     {
00256         std::size_t new_cap = _capacity ? _capacity * 2 : 1; // rezervuojame apie 1.5 kartus daugiau
00257         reserve(new_cap);
00258     }
00259
00260     _data[_size] = new_value;
00261     _size++;
00262 }
00263
00264 void push_back(T&& new_value)
00265 {
00266     if(_size == _capacity)
00267     {
00268         std::size_t new_cap = _capacity ? _capacity * 2 : 1;
00269         reserve(new_cap); // rezervuojame apie 1.5 kartus daugiau
00270     }
00271
00272     _data[_size] = std::move(new_value);
00273     _size++;
00274 }
00275
00276 void pop_back()
00277 {
00278     if (_size == 0)
00279         throw std::out_of_range("Vector::pop_back(): tuscias Vektorius");
00280
00281     std::destroy_at(&_data[_size - 1]); // sunaikiname paskutinį elementą
00282     --_size;                             // sumažiname dydį
00283 }
00284
00285 void clear()
00286 {
00287     for(size_t i = 0; i < _size; i++)
00288     {
00289         std::destroy_at(&_data[i]);
00290     }
00291     _size = 0;
00292 }
00293
00294 void swap(Vector<T> &naujas)
00295 {
00296     std::swap(this->_data, naujas._data);
00297     std::swap(this->_size, naujas._size);
00298     std::swap(this->_capacity, naujas._capacity);
00299 }
00300
00301 void sort()
00302 {
00303     std::sort(begin(), end());
00304 }
00305
00306 void assign(size_t count, const T& value) {
00307     if (count > _capacity)
00308         reserve(count);
00309
00310     // sunaikinam senus objektus
00311     for (size_t i = 0; i < _size; ++i)
00312         std::destroy_at(_data + i);
00313
00314     // konstruojam naujus
00315     for (size_t i = 0; i < count; ++i)
00316         std::construct_at(_data + i, value);
00317
00318     _size = count;

```



```

00319 }
00320
00321 iterator end() noexcept { return _data + _size; } // tas pats, kas &_data[_size]
00322
00323 const_iterator end() const { return _data + _size; } // tas pats, kas &_data[_size]
00324
00325 const_iterator cend() const noexcept { return _data + _size; } // tas pats, kas &_data[_size]
00326
00327 iterator begin() noexcept { return _data; }
00328
00329 const_iterator begin() const noexcept { return _data; }
00330
00331 const_iterator cbegin() const noexcept { return _data; }
00332
00333 reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00334
00335 const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00336
00337 reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00338
00339 const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00340
00341 const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00342
00343 const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00344
00345 iterator insert(const_iterator pos, const T& value)
00346 {
00347     const std::size_t idx = static_cast<std::size_t>(pos - begin());
00348     if (pos < begin() || pos > end()) {
00349         throw std::out_of_range("insert position is invalid");
00350     }
00351     if (_size == _capacity)
00352         reserve(_capacity ? _capacity * 2 : 1);
00353     for(size_t i = _size; i > idx; i--)
00354     {
00355         std::construct_at(&_data[i], std::move(_data[i-1]));
00356         std::destroy_at(&_data[i-1]);
00357     }
00358     std::construct_at(_data + idx, value);
00359     ++_size;
00360     return _data + idx; // reikia iteratoriaus
00361 }
00362
00363 iterator insert( const_iterator pos, T&& value )
00364 {
00365     const std::size_t idx = static_cast<std::size_t>(pos - begin());
00366     if (pos < begin() || pos > end()) {
00367         throw std::out_of_range("insert position is invalid");
00368     }
00369     if (_size == _capacity)
00370         reserve(_capacity ? _capacity * 2 : 1);
00371     for(size_t i = _size; i > idx; i--)
00372     {
00373         std::construct_at(&_data[i], std::move(_data[i-1]));
00374         std::destroy_at(&_data[i-1]);
00375     }
00376     std::construct_at(_data + idx, std::move(value));
00377     ++_size;
00378     return _data + idx;
00379 }
00380
00381 T* data() noexcept
00382 {
00383     return _data; // Pirmo _data elemento adresas
00384 }
00385
00386 const T* data() const
00387 {
00388     return _data; // Pirmo _data elemento adresas
00389 }
00390
00391 template <typename... Args>
00392 iterator emplace(const_iterator pos, Args&&... args)
00393 {
00394     std::size_t idx = static_cast<std::size_t>(pos - begin());
00395     if (_size == _capacity)
00396         reserve(_capacity ? (_capacity * 3) / 2 : 1);
00397     /* pastumiame elementus vietos užleidimui */
00398     for (std::size_t i = _size; i-- > idx; ) {
00399         std::construct_at(_data + i + 1, std::move(_data[i]));
00400     }
00401     std::construct_at(_data + idx, ...args);
00402     ++_size;
00403     return _data + idx;
00404 }

```

```

00406         std::destroy_at (_data + i);
00407     }
00408
00409     /* sukonstruojame elementą vietoje */
00410     std::construct_at(_data + idx, std::forward<Args>(args)...);
00411
00412     ++_size;
00413     return _data + idx;
00414 }
00415
00416 template <typename... Args>
00417 void emplace_back( Args&&... args )
00418 {
00419     if(_capacity == _size)
00420         reserve(_capacity ? _capacity * 2 : 1);
00421
00422     std::construct_at(&_amp;_data[_size], std::forward<Args>(args)...);
00423
00424     ++_size;
00425 }
00426
00427 template <typename InputIt>
00428 iterator insert_range(const_iterator pos, InputIt first, InputIt last)
00429 {
00430     std::size_t idx = static_cast<std::size_t>(pos - begin()); // fiksuojame dar galiojantį
00431     std::size_t count = std::distance(first, last);
00432     if (count == 0) return _data + idx; // nieko įterpti
00433
00434     if (_size + count > _capacity) // galimas _data perkėlimas!
00435         reserve(_capacity ? (_capacity * 3) / 2 : count); // 1.5x, bent count
00436
00437     /* 1) pastumiame senus elementus į dešinę (nuo galo, kad nepersirašytų) */
00438     for (std::size_t i = _size; i-- > idx; ) {
00439         std::construct_at(_data + i + count, std::move(_data[i]));
00440         std::destroy_at (_data + i);
00441     }
00442
00443     /* 2) įterpiame naujus */
00444     for (std::size_t i = 0; i < count; ++i, ++first) {
00445         std::construct_at(_data + idx + i, *first);
00446     }
00447
00448     _size += count;
00449     return _data + idx;
00450 }
00451
00452 template <typename R>
00453 constexpr void assign_range(R&& rg)
00454 {
00455     static_assert(std::ranges::input_range<R>, "R must be an input range");
00456     static_assert(std::constructible_from<T, std::ranges::range_reference_t<R>, "T must be
constructible from range reference type");
00457
00458     size_t count = std::ranges::distance(rg);
00459     if (count > _capacity)
00460         reserve(count + count / 2);
00461
00462     auto it = std::ranges::begin(rg);
00463     for(size_t i=0; i < count; i++, it++)
00464     {
00465         std::destroy_at(&_amp;_data[i]);
00466         std::construct_at(&_amp;_data[i], *it);
00467     }
00468
00469     if(count<_size)
00470         for(size_t i = count; i < _size; i++)
00471         {
00472             std::destroy_at(&_amp;_data[i]);
00473         }
00474
00475     _size = count;
00476 }
00477
00478 allocator_type get_allocator() const
00479 {
00480     return alloc;
00481 }
00482
00483 template <typename R>
00484 constexpr void append_range(R&& rg)
00485 {
00486     static_assert(std::ranges::input_range<R>, "R must be an input range");
00487     static_assert(std::constructible_from<T, std::ranges::range_reference_t<R>, "T must be
constructible from range reference type");
00488
00489     const size_t count = std::ranges::distance(rg);
00490

```

```

00491     if (_size + count > _capacity)
00492         reserve(_size + count + count / 2);
00493
00494     auto it = std::ranges::begin(rg);
00495     for (size_t i = 0; i < count; ++i, ++it)
00496     {
00497         std::construct_at(&_data[_size + i], std::forward<decltype(*it)>(*it));
00498     }
00499
00500     _size += count;
00501 }
00502
00503 friend bool operator==(const Vector<T>& a, const Vector<T>& b)
00504 {
00505     if (a._size != b._size)
00506         return false;
00507     for (size_t i = 0; i < a._size; ++i) {
00508         if (a._data[i] == b._data[i]) return false;
00509     }
00510     return true;
00511 }
00512
00513 friend bool operator!=(const Vector<T>& a, const Vector<T>& b) {
00514     return !(a == b);
00515 }
00516
00517 iterator erase(const_iterator pos)
00518 {
00519     size_t index = pos - begin();
00520     if (pos < begin() || pos >= end()) {
00521         throw std::out_of_range("erase position is invalid");
00522     }
00523
00524     for (size_t i = index + 1; i < _size; ++i)
00525     {
00526         _data[i-1] = std::move(_data[i]);
00527     }
00528
00529     std::destroy_at(&_data[_size - 1]); // sunaikiname paskutinį elementą
00530     _size--;
00531
00532     return _data + (pos - _data);
00533 }
00534
00535 iterator erase(const_iterator first, const_iterator last)
00536 {
00537     if (first < begin() || first > end() || last < begin() || last > end() || last < first)
00538     {
00539         throw std::out_of_range("Vector::erase(range) invalid range");
00540     }
00541
00542     // Apskaičiuojam, kiek elementų triname
00543     size_t idx = first - begin();
00544     size_t count = last - first; // gali būti 0
00545
00546     for (size_t i = idx + count; i < _size; ++i) {
00547         _data[i - count] = std::move(_data[i]);
00548     }
00549
00550     for (size_t i = _size - count; i < _size; ++i) {
00551         std::destroy_at(_data + i);
00552     }
00553
00554     _size -= count;
00555
00556     return _data + idx;
00557 }
00558
00559 template <class U>
00560 friend std::ostream& operator<<(std::ostream& os, const Vector<U>& vec);
00561 };
00562
00563 template <class U>
00564 std::ostream& operator<<(std::ostream& os, const Vector<U>& vec)
00565 {
00566     for (const auto& el : vec) os << el;
00567     return os;
00568 }
00569 #endif

```

6.7 zmogus.h

```
00001 #ifndef ZMOGUS_H
```

```
00002 #define ZMOGUS_H
00003
00004 #include "main.h"
00005
00006 // Bazine (abstrakti) klase Zmogus is kurios darysime isvestine klase Studentas
00007 class Zmogus{
00008
00009     protected:
00010         std::string vardas, pavarde;
00011
00012     public:
00013         Zmogus() : vardas(""), pavarde(""){} // Default konstruktorius
00014         Zmogus(string vardas, string pavarde); // Konstruktorius
00015         virtual ~Zmogus(); // Destruktorius
00016
00017         string GetVardas() const; // Grazina varda
00018         string GetPavarde() const; // Grazina pavarde
00019         void SetVardas(string vardas); // Nustato varda
00020         void SetPavarde(string pavarde); // Nustato pavarde
00021         int GetVardasSize() const; // Grazina vardo dydi
00022         int GetPavardeSize() const; // Grazina pavardes dydi
00023
00024         virtual void Spausdinti(std::ostream& os) const = 0; // Si funkcija padaro klase Zmogus
00025         abstrakcia, nes ji neturi implementacijos
00026 };
00027 #endif
```

Index

Pasirinkimas, [23](#)

Spausdinti

Studentas, [25](#)

Studentas, [23](#)

Spausdinti, [25](#)

Timer, [25](#)

Vector< T >, [25](#)

Versijų aprašymai:, [2](#)

Zmogus, [27](#)